

Bring Your Own Problem - Pre-Work Brief

In ML Systems Week your group will take **one** machine-learning problem and carry it the whole way: frame it, design a system around it, and **defend that design in a live viva** - which is what you're assessed on. Building a pipeline for it with your LLM assistant is an *optional stretch* on top, not required. You get to **choose that problem yourselves** - and this brief is how you do it well.

Work on this with your assigned group. One problem per group, chosen together and submitted together - not one each. Settling on it as a group *is* part of the exercise: you'll have to argue for a shape, hear the objections, and converge - which is the same muscle the viva tests. In the viva, questions go to **individuals**, so everyone needs to be able to defend the choice, not just whoever suggested it.

Choosing a good problem is not a warm-up; it *is* the first piece of judgement we're teaching. Shape matters: some of the design questions (cost of error, leakage, drift, monitoring) only *have answers* for certain kinds of problem, so a poorly-shaped choice leaves your group with less to reason about. The bar in Section 2 is deliberately broad - a short list your problem **must** have, and a wider pool where **4 of 6** is enough. Use the time before the week starts to land on one together, and submit it for approval.

If your group can't land a problem you're confident in, that's fine - tell us and we'll assign you one from our scenario library. No group starts the week without a vetted problem.

1. What "good" looks like (aim here)

The sweet spot for this week is:

A supervised classification / triage model that supports a human decision, case by case.

In plain terms: your model looks at one case at a time - a transaction, a customer, a loan application, a machine, a support ticket - and for each one predicts something that helps a person **decide what to do**: approve it, review it, flag it as risky, prioritise it, or step in. A human is in the loop, acting on the model's output (at least for the cases it flags). The two errors don't cost the same, and the right call isn't obvious - that's what makes it worth reasoning about.

That's the shape that gets the most out of the week's material - not a template your problem has to match line for line. Section 2 is the actual bar; everything in this section is orientation.

Two things you don't need.

A real dataset. You'll generate a small synthetic one with your LLM during the week, so choose your problem on its *shape*, not on whether you can get hold of the data. "We can't access that data" is never a reason to rule an idea out.

Time-series or forecasting data. You classify each case on its own merits. It *helps* if your cases have some sense of before-and-after - records that accumulate, outcomes that land later - because it makes drift and delayed labels concrete rather than hypothetical. But that's an enrichment, not a requirement, and emphatically **not** a request for a forecasting problem, sequential features, or a time index. See Section 2B.

The same shape shows up in many forms. A few, to widen your thinking beyond any one domain:

- **Risk / fraud / abuse** - flag the transactions, sellers, or claims worth a closer look.
- **Failure / churn / complaint** - predict which customers, machines, or accounts are heading for trouble, so someone intervenes early.
- **Quality / eligibility screening** - approve, reject, or escalate an application, submission, or item.
- **Prioritising scarce expert attention** - who or what gets the limited inspection, audit, outreach, review, or maintenance slot this week.
- **Surfacing opportunity, not just risk** - which lead, listing, or case is worth someone's time. The systems thinking is identical: you're prioritising scarce human attention, whether it points at trouble or upside.

The **two example scenarios released alongside this brief** (the *Example Scenarios* document) are just *two instances* from the risk-and-screening end of that list - marketplace-trust triage and luxury-authentication triage. **Yours should come from a domain someone in your group knows, and can look quite different**; what matters is that it meets the requirements in Section 2, not that it resembles the examples. Read both as a group and map each onto the Section 2 checklist - it's the quickest way to calibrate what the bar actually feels like.

2. The requirements your problem must meet

Run your idea through this. There are **two tiers**:

- a **non-negotiable core** your problem must meet *in full* - the three requirements in Section 2A, plus the two gates in Section 2C and Section 2D: **five in all**;
- an **enrichment pool** in Section 2B, where you need **at least 4 of the 6**.

Miss a **core** requirement and you'll need to reshape the idea or pick another. For any *enrichment* line you don't hit, just be ready to say why it's light in your problem - that reasoning is itself a valid answer in the

week. And **don't chase a problem that maxes out every line**: the shape in Section 1 drags most of the enrichment in on its own.

Quick glossary. Several terms below are exactly what the ML Systems week teaches - here's a plain-language gloss so you can use this checklist *now*, before you've covered them:

- *Supervised* - you have labelled examples to learn from: each past case comes with a known answer.
- *Label* - the answer you're predicting for each case ("did this ticket miss its deadline?").
- *Triage* - sorting incoming cases by how much attention each one needs.
- *ML task type* - what kind of prediction it is: classification (which category?), ranking (in what order?), regression (what number?), and so on.
- *Success metric* - the number you'd judge the model by - ideally one that means something in business terms, not just model terms.
- *Baseline* - the simple approach your model has to beat to be worth building; usually a handful of business rules.
- *Class imbalance* - the outcome you care about is rare, so plain accuracy can look great while missing the point.
- *Data leakage* - accidentally using information you wouldn't actually have at prediction time, which makes the model look better than it really is. It also covers the same entity (the same seller, machine, or customer) turning up in both your training and test data, so the model has effectively already seen the answer.
- *Train/serve skew* - a feature computed one way while training and a slightly different way when live, so the model misbehaves in production.
- *Data drift* - the inputs shift over time; *concept drift* - the input-to-outcome relationship itself shifts.
- *Feedback loop* - acting on a prediction changes the outcome you later measure.
- *Serving pattern* - how predictions are delivered: a scheduled batch job vs a live, on-demand API call.
- *Proxy label / selection bias* - your label is a stand-in for what you truly care about, or you only see outcomes for some cases (e.g. only the ones a human reviewed).

A. Non-negotiable core - all required

Miss one of these and a whole part of the week has nothing to work with, so these are hard requirements - no partial credit.

- **Supervised prediction with a definable label.** Classification / triage into tiers (or an explicitly-framed *ranking of what to attend to first*). You have, or can define, a **label** - the thing you're predicting. *Without this there's no training, evaluation, or metric at all.*
- **A named human acts on the model's output** - at least on the cases it flags. Someone approves, reviews, prioritises, or intervenes; the model *supports* a decision rather than running fully

autonomously. ("Send this one to a human" is a valid output.) *This is where the deployment, monitoring, and human-factors reasoning lives.*

- **Errors cost different amounts.** One kind of mistake is clearly worse than the other. (Bonus if the cost also *varies case to case* - a high-value account, a safety-critical case.) *Symmetric errors leave the cost-of-error and economics reasoning - a heavily-weighted part of the assessment - with nothing to bite on.*

The other two core requirements are the gates below: **Section 2C** (it genuinely needs ML) and the **one model, one decision** rule in **Section 2D**. Section 5 lists all five together.

B. Enrichment - at least 4 of these 6

These make the design *rich*, and no single one needs to be strong. **4 of the 6 is a genuine pass, not a bare one** - don't go hunting for a better problem just because you aren't at five or six. A problem with genuinely *none* of them is too thin, but trying to force all six into one clean problem is what makes choosing feel impossible. (You'll name these precisely during the week; for now, sanity-check each against the glossary above.)

- **Class imbalance** - the critical class is rare, so plain accuracy would mislead.
- **An imperfect label** - it arrives *late*, is *noisy*, is a *proxy* for what you truly care about, or you only observe it for a *biased subset* of cases.
- **Some time dimension to the data** - cases accumulate and outcomes land after the fact, so you can split by time. **Not a time-series or forecasting requirement:** you still classify each case on its own, with no sequential features or time index. Most problems have this simply because records pile up over months. *A fixed snapshot is fine too* - say so, and reason about what drift **would** look like.
- **A data-leakage trap and/or train/serve skew** - a field known only *after* the outcome, an entity that bleeds across train/test, or a feature that could be computed differently offline vs live.
- **A feedback loop and/or a genuine serving-pattern choice** - acting on a prediction changes the outcome you later measure; or batch-vs-real-time isn't obvious and is dictated by how the prediction is used.
- **A plausible data-drift and/or concept-drift story** - the inputs could shift, and/or the input-to-outcome relationship could shift. (Both is nice; one clear example is enough for this line.)

C. The "do you even need ML?" gate

Before committing: could a handful of business rules solve this acceptably? If yes, it's the wrong problem for this exercise. Reach for ML only where the pattern is too complex to hand-specify and there's data to learn it from. (That rule-based version becomes your **baseline** later, so keep it in mind.)

D. Disqualifiers - if your idea is one of these, pick another

- **Unsupervised / clustering** ("segment our customers") - no labels, no false-positive/negative, no cost matrix. Guts the evaluation and economics material.

- **A pure generative-AI / chatbot / content-generation product** - evaluation, labels, drift, and cost-of-error don't map here.
- **Full automation with no human** - kills the human-in-the-loop and half the monitoring/feedback richness.
- **Symmetric, low-stakes errors** - nothing for the cost-of-error reasoning to bite on.
- **A problem where the ML spec was handed to you** - a Kaggle-style competition, a tutorial dataset with the target and metric already defined, or a task someone else already scoped. The framing work is what's being assessed, so it has to be yours. (*Framing it yourselves for the Section 4 proposal is exactly right - that's the exercise, not a violation of it.*)
- **A platform of many models.** Scope to **one model, one decision** - a good system, not a big one.

Example: A pure ranking or recommendation idea ("recommend products to each user") fails as-is - no human-in-the-loop triage, no cost asymmetry. But it can often be *reshaped* into one that passes: "help a live-stream host decide which product to feature next, given they can only push a few" is the same data framed as *supporting a human's decision about where to focus*. If your idea is close but missing the human or the asymmetry, look for that reframing before you discard it.

Note: Being light on any single enrichment line is fine. Being blank on **both** drift/monitoring **and** leakage/skew *at the same time* is not - that combination leaves too little for the viva to probe. Reshape it.

3. How to find one (you have one and a half weeks - use them)

You don't need a novel idea; you need a well-shaped one. **Start by pooling what the group already knows** - go round and have everyone name a domain they've actually worked in or understand well, then pick the one with the best shape. A group has several people's experience to draw on; use it before inventing something from scratch. Good hunting grounds:

- **A real pain someone in the group has seen** - an internship or past job where people made the same call case by case (approve / reject / flag / prioritise / step in), and some calls were expensive to get wrong.
- **A public domain one of you understands** - lending, insurance, logistics, healthcare, predictive maintenance, fraud, hiring, customer retention. Anywhere a person repeatedly decides what to do with individual cases, and some decisions cost more than others.

Then **pressure-test it with your LLM** - which is exactly the muscle this week trains:

"Here are the requirements for a machine-learning problem [paste Section 2]. Here's our idea: [describe it]. Does it meet all five core requirements (Section 2A, plus the Section 2C and 2D gates)? Which of the enrichment lines (Section 2B) does it plausibly hit, and which are light? For anything it misses, is there a small reshape that fixes it - or is it reasonable to leave light and explain why?"

Iterate until it clears the bar - **not** until it ticks every box; that isn't what's being asked, and chasing it is how groups burn the whole runway and still feel stuck. If two or three ideas all fail on the same *core* line (usually "no human in the loop" or "errors are symmetric"), that's a signal you're drawn to a shape that doesn't fit - switch hunting grounds.

4. What to submit, and when

Submit **one problem proposal per group, before ML Systems Week starts**, so your mentor can approve it or steer you first. Keep it to half a page:

- **Problem statement (2-3 sentences)** - the objective, the ML task type, the target, the success metric, the baseline to beat, and **who acts** on the output. (This is the same framing you'll expand into the one-page design blueprint - Section 10.2 of the P1 notebook - during the week, so pin down as much of it now as you can.)
- **Cost-of-error asymmetry** - which mistake is worse, and roughly by how much.
- **Data plan** - you'll synthesise a small dataset with your LLM during the week; note the rough fields you'd generate. (Realism isn't graded; a plausible shape is enough.)
- **Requirements check** - confirm your problem meets all five **core** requirements (Section 2A plus the Sections 2C/2D gates) and **at least 4 of the 6** enrichment lines (Section 2B); for any enrichment lines it's light on, note in a sentence why (that's a valid answer, not a gap). Rough is fine; you'll sharpen these in the week.

Your assigned mentor signs off on the problem **and** its framing. A miscounted checklist isn't a failure - if the problem is sound, they'll correct the count and approve it. The **one** round of feedback is for when the *problem itself* needs reshaping; if it still doesn't clear after that, you'll run the week on a library scenario instead - no drama, but the choice stops being yours.

5. Final self-check (run before you submit)

Core - all five must be true:

- It's supervised classification / triage with a definable label.
- A named human acts on the model's output (at least on the cases it flags).
- One error is clearly costlier than the other.
- It genuinely needs ML (business rules wouldn't do).
- It's one model, one decision - not a platform.

Enrichment - at least 4 of these 6 must be true; for any that aren't, we can say why it's light here:

- The critical class is rare / imbalanced.
- The label is late, noisy, a proxy, or only seen for some cases.

- There's some time dimension to the data. (If it's a snapshot, this line is *light* - say so, and reason about what drift would look like instead. It needn't be a time-series/forecasting problem, and never needs sequential features.)
- We can point to a leakage trap and/or a train/serve skew.
- We can point to a feedback loop and/or a genuine batch-vs-real-time choice.
- There's a plausible data-drift and/or concept-drift story.

And the one Section 2D disqualifier the core checks don't already catch:

- The framing is ours - nobody handed us the ML spec (no Kaggle-style task with the target and metric already defined).

Tick **all five core**, **at least 4 of the 6 enrichment**, and the Section 2D box - and make sure you aren't blank on *both* drift/monitoring and leakage/skew - and you've got a problem that will carry you through the whole week.